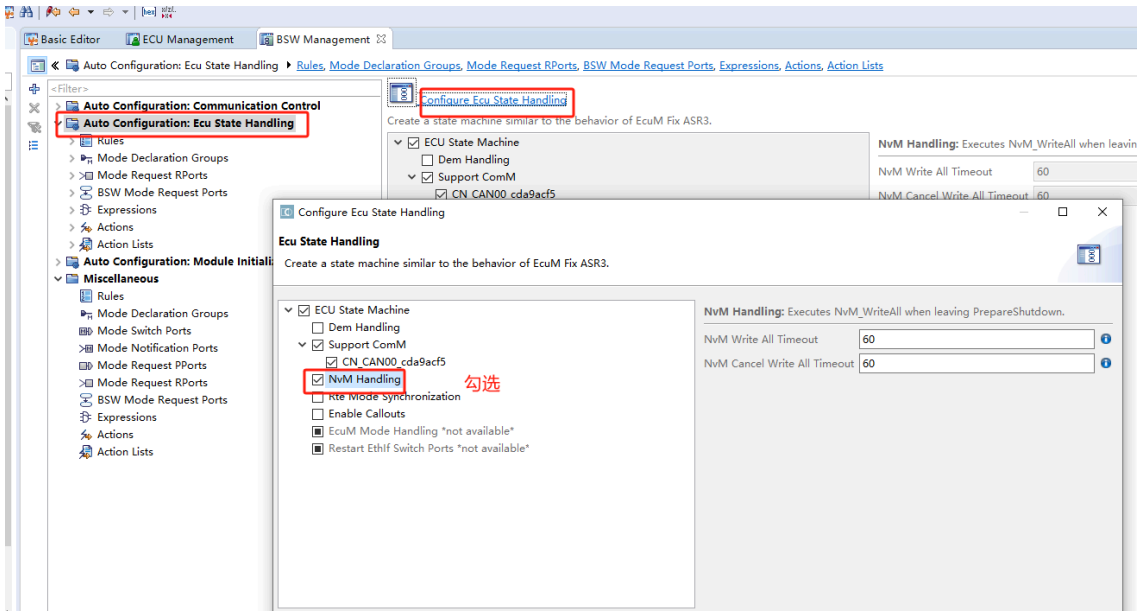
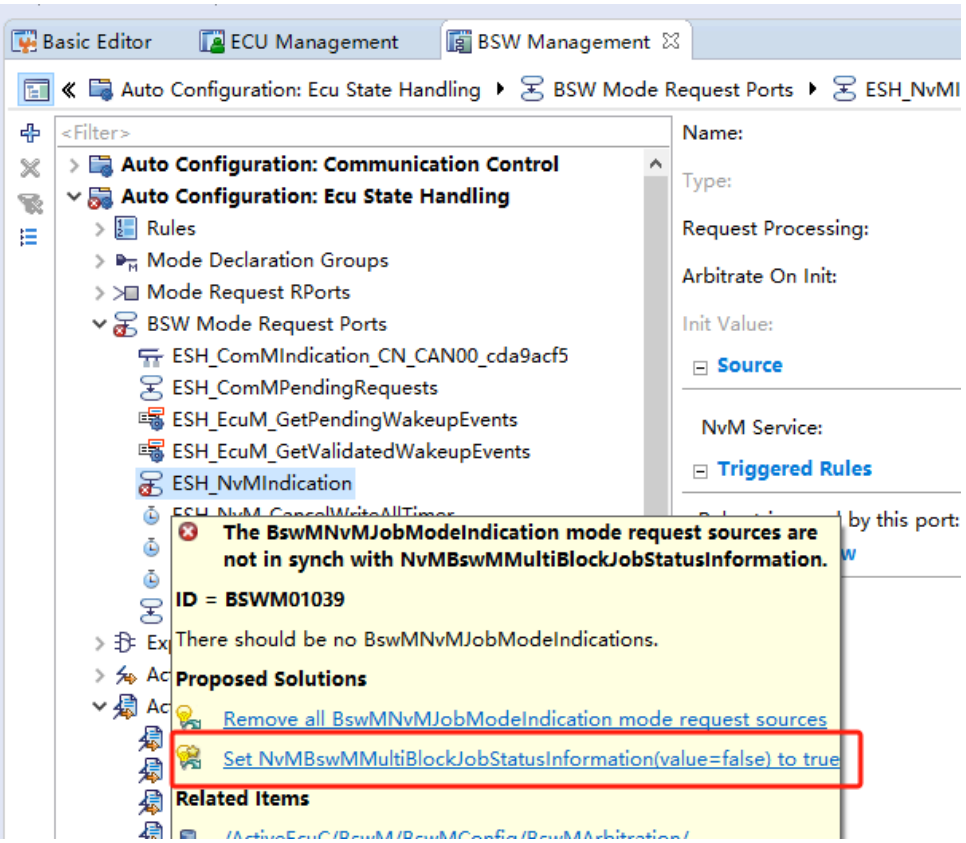
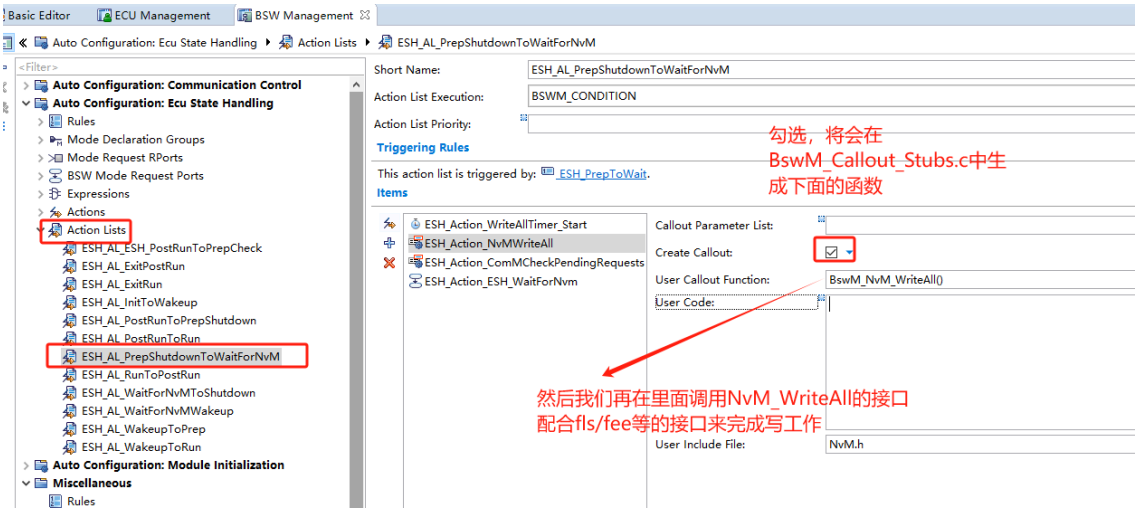
28、休眠前的NvM_WriteAll配置

卖家'漩涡遨游aa' 专属资料，非在漩涡遨游aa 购买资料，不提供答疑(一次购买仅提供有限次数10次答疑)

一、BswM配置







因为调用NvM_WriteAll前，需要调用NvM_SetRamBlockStatus 函数，去改变需要休眠前存储的block状态，否则即将休眠前调用NvM_WriteAll 后不存储！

4.4.7 NvM_SetRamBlockStatus

Prototype

```
Std_ReturnType NvM_SetRamBlockStatus ( NvM_BlockIdType BlockId,
                                       boolean BlockChanged )
```

Parameter

BlockId	The block identifier.
BlockChanged	Sets the new status of the RAM block: TRUE: Validates the RAM block and marks it as changed. If the block has a CRC and the option NVM_CALC_RAM_BLOCK_CRC is TRUE the CRC calculation is initiated. FALSE: Mark the block as unchanged

Return code

E_OK	Request has been accepted.
E_NOT_OK	Request has not been accepted, e.g. Det error occurred.

Functional Description

The request sets a block's status to valid/changed respectively to unchanged. Setting a block to changed marks it for writing it during NvM_WriteAll().

If the block shall be set to **changed**, it has a CRC and the option NVM_CALC_RAM_BLOCK_CRC is TRUE the CRC calculation of the RAM block is initiated.

Note

Though this service is defined to operate synchronously, the CRC re-calculation will be performed asynchronously. However, there is no restriction on accessing RAM block data, or on calling other services. Consistency of data and CRC is ensured by WriteBlock/WriteAll, which will unconditionally recalculate the CRC before writing. Requesting CRC re-calculation, using NvM_SetRamBlockStatus again, will be recognized in a save way, the calculation will be re-queued, if necessary.

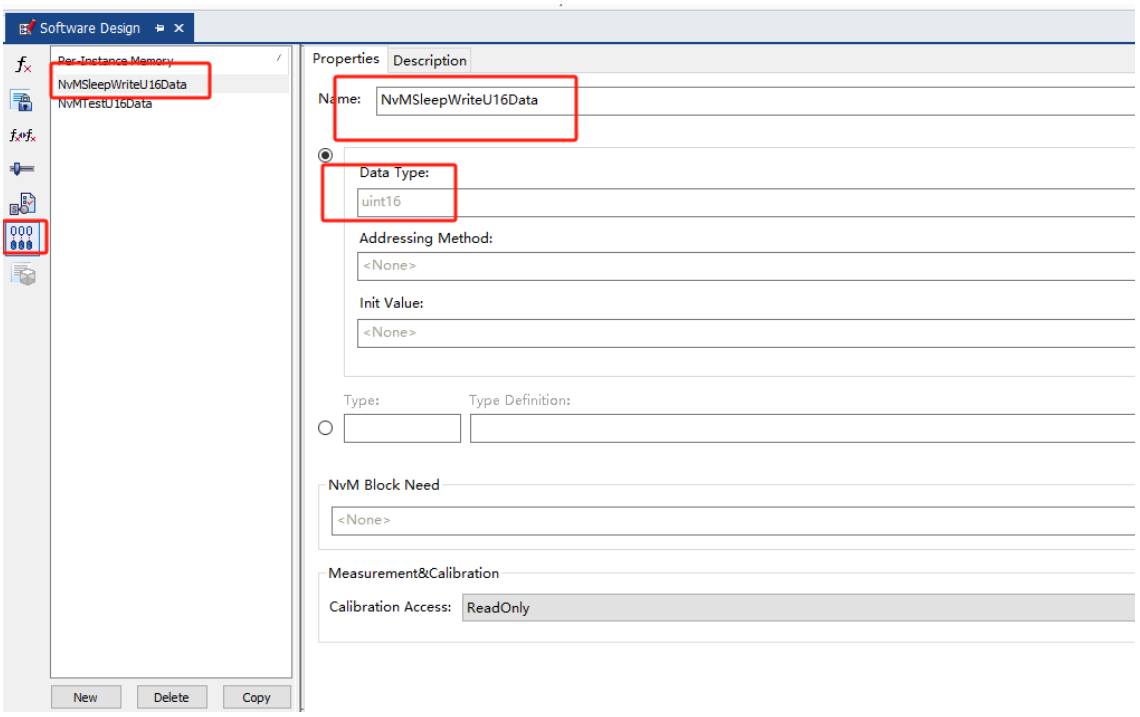
添加下面代码

```
FUNC(void, BSWM_CODE) BswM_NvM_WriteAll(void)
{
/**********************************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           <USERBLOCK BswM_NvM_WriteAll>                    DO NOT CHANGE THIS COMMENT!
 **********************************************************************************************************************/
  NvM_RequestResultType Int_NvMultiBlockStatus = NVM_REQ_NOT_OK;
  Std_ReturnType ret = E_NOT_OK;
  NvM_WriteAll();

  do
  {
      NvM_MainFunction();
      Fee_MainFunction();
      Fls_MainFunction();
      ret = NvM_GetErrorStatus(NvMConf___MultiBlockRequest, &Int_NvMultiBlockStatus);

  #if defined(VGEN_ENABLE_DRVWD)
    Wdg_Trigger();
  #endif
  } while ((Int_NvMultiBlockStatus == NVM_REQ_PENDING)&&( ret == E_OK ));
/**********************************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!           </USERBLOCK>                                     DO NOT CHANGE THIS COMMENT!
 **********************************************************************************************************************/

} /* End of BswM_NvM_WriteAll */
```

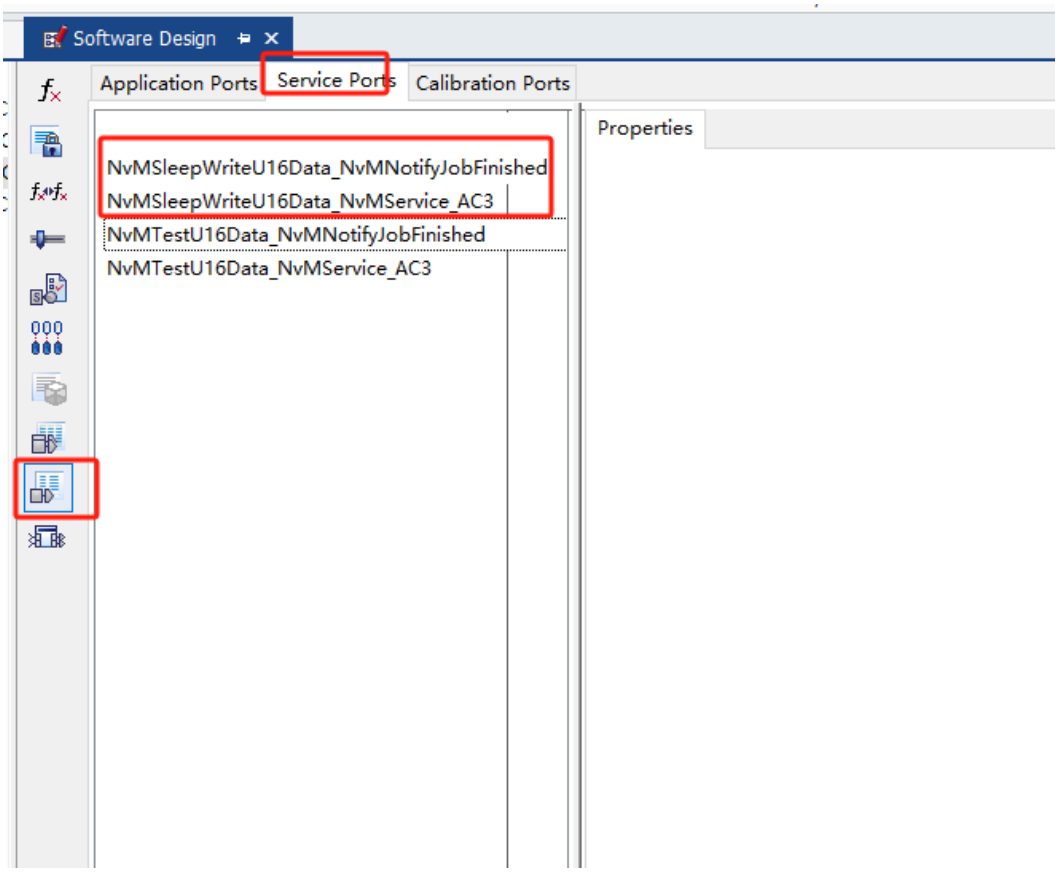
二、新建NvM Block 测试

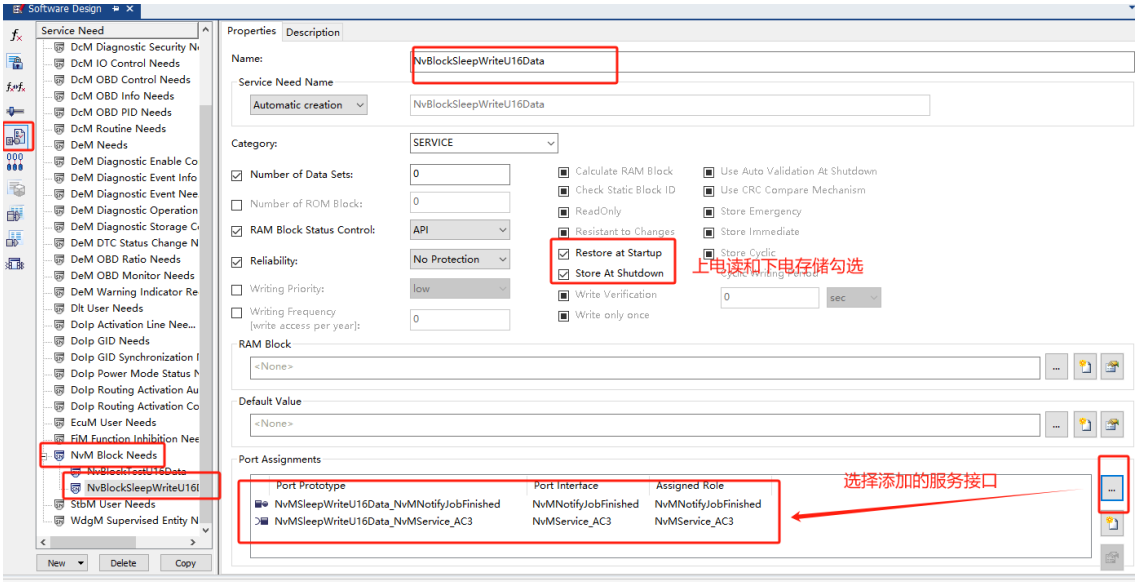
2.1、developer添加配置块

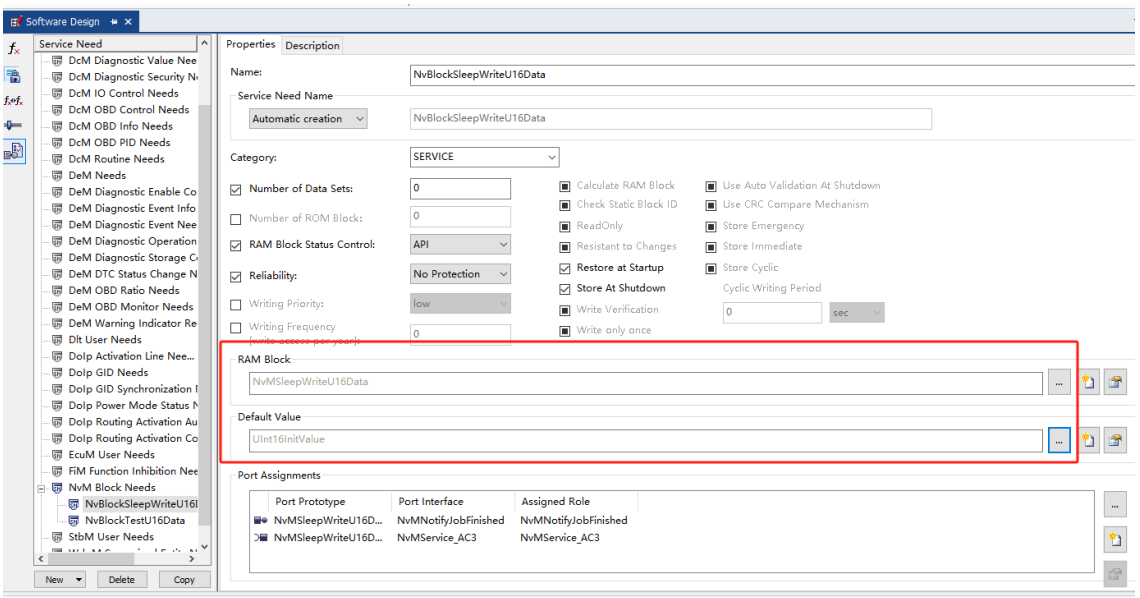
打开developer，在CtApNvMSwc 中添加

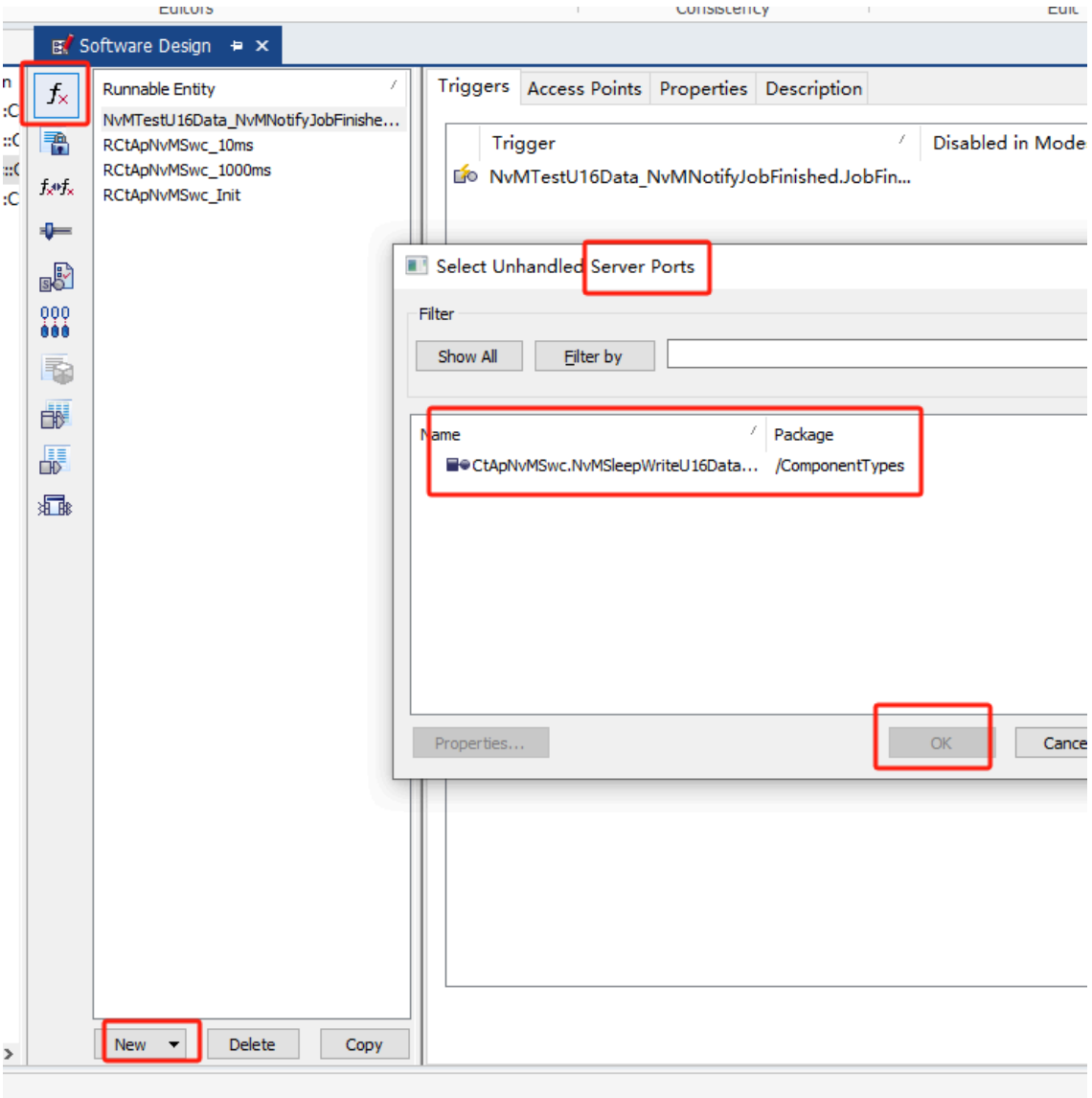


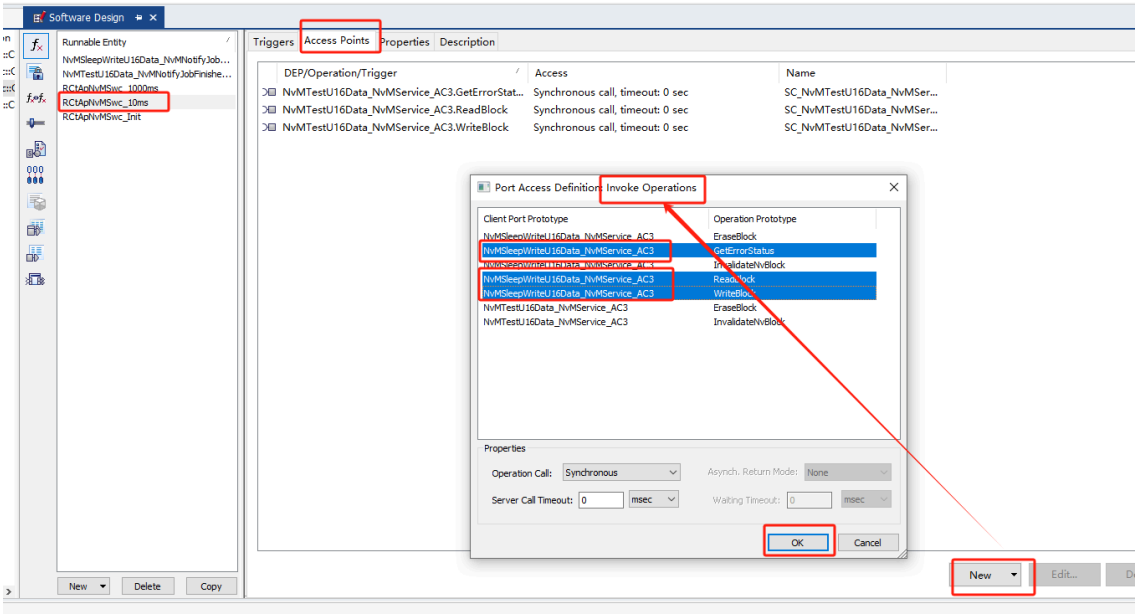
按照原来的复制，添加这两个Service ports


Software Design
Application Ports
Service Ports
Calibration Ports
Properties
NvMSleepWriteU16Data_NvMNotifyJobFinished
NvMSleepWriteU16Data_NvMService_AC3
NvMTestU16Data_NvMNotifyJobFinished
NvMTestU16Data_NvMService_AC3


Software Design
Service Need
DcM Diagnostic Security N
DcM IO Control Needs
DcM OBD Control Needs
DcM OBD Info Needs
DcM OBD PID Needs
DcM Routine Needs
DeM Needs
DeM Diagnostic Enable Co
DeM Diagnostic Event Info
DeM Diagnostic Event Nee
DeM Diagnostic Operation
DeM Diagnostic Storage C
DeM DTC Status Change N
DeM OBD Ratio Needs
DeM OBD Monitor Needs
DeM Warning Indicator Re
Dlt User Needs
DoIp Activation Line Nee...
DoIp GID Needs
DoIp GID Synchronization
DoIp Power Mode Status N
DoIp Routing Activation Au
DoIp Routing Activation Co
EcuM User Needs
FiM Function Inhibition Nee
NvM Block Needs
NvBlockSleepWriteU16
StbM User Needs
WdgM Supervised Entity N
New
Delete
Copy
Properties
Description
Name:
NvBlockSleepWriteU16Data
Service Need Name
Automatic creation
NvBlockSleepWriteU16Data
Category:
SERVICE
Number of Data Sets:
Number of ROM Block:
RAM Block Status Control:
API
Reliability:
No Protection
Writing Priority:
low
Writing Frequency
(write access per year):
Calculate RAM Block
Check Static Block ID
ReadOnly
Resistant to Changes
Restore at Startup
Store At Shutdown
Write Verification
Write only once
Use Auto Validation At Shutdown
Use CRC Compare Mechanism
Store Emergency
Store Immediate
Store Cyclic
上电读和下电存储勾选
sec
RAM Block
<None>
Default Value
<None>
Port Assignments
Port Prototype
Port Interface
Assigned Role
NvMSleepWriteU16Data_NvMNotifyJobFinished
NvMNotifyJobFinished
NvMNotifyJobFinished
NvMSleepWriteU16Data_NvMService_AC3
NvMService_AC3
NvMService_AC3
选择添加的服务接口


Software Design
Service Need
Properties
Description
Name:
NvBlockSleepWriteU16Data
Service Need Name
Automatic creation
Category:
SERVICE
Number of Data Sets:
Number of ROM Block:
RAM Block Status Control:
API
Reliability:
No Protection
Writing Priority:
Writing Frequency
Calculate RAM Block
Check Static Block ID
ReadOnly
Resistant to Changes
Restore at Startup
Store At Shutdown
Write Verification
Write only once
Use Auto Validation At Shutdown
Use CRC Compare Mechanism
Store Emergency
Store Immediate
Store Cyclic
Cyclic Writing Period
RAM Block
NvMSleepWriteU16Data
Default Value
UInt16InitValue
Port Assignments
Port Prototype
Port Interface
Assigned Role
NvMNotifyJobFinished
NvMService_AC3
New
Delete
Copy


Software Design
Runnable Entity
NvMTestU16Data_NvMNotifyJobFinishe...
RCtApNvMSwc_10ms
RCtApNvMSwc_1000ms
RCtApNvMSwc_Init
Triggers
Access Points
Properties
Description
Trigger
Disabled in Mode
NvMTestU16Data_NvMNotifyJobFinished.JobFin...
Select Unhandled Server Ports
Filter
Show All
Filter by
Name
Package
CtApNvMSwc.NvMSleepWriteU16Data...
/ComponentTypes
Properties...
OK
Cance
New
Delete
Copy

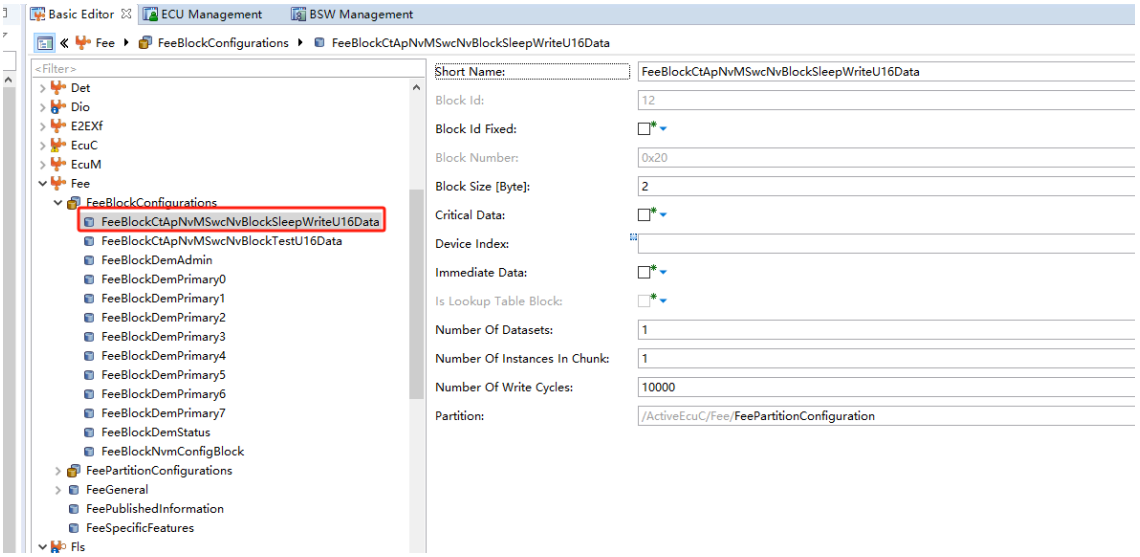


保存同步到cfg

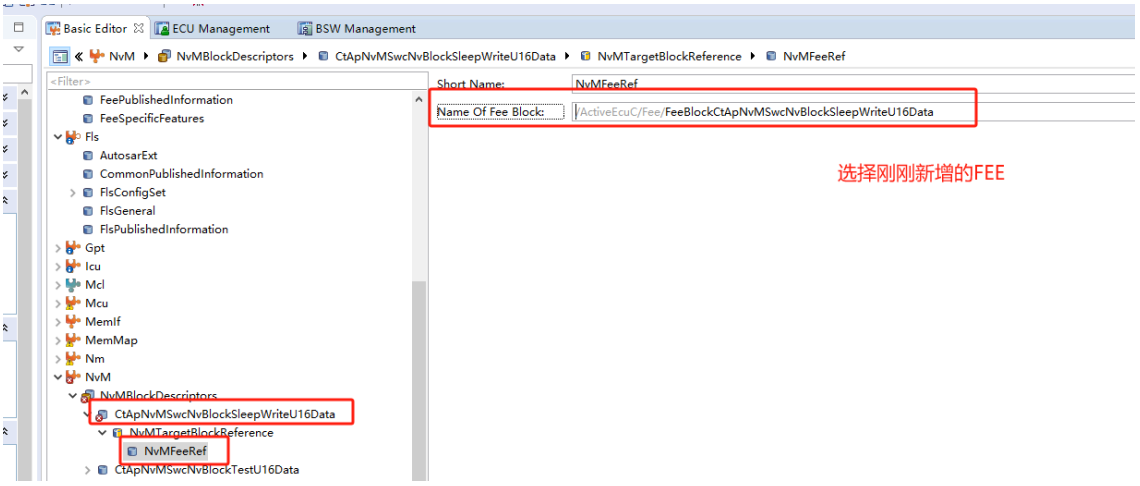
2.2、cfg 补充相关NvM的配置

同步下

Fee新建块



NvM

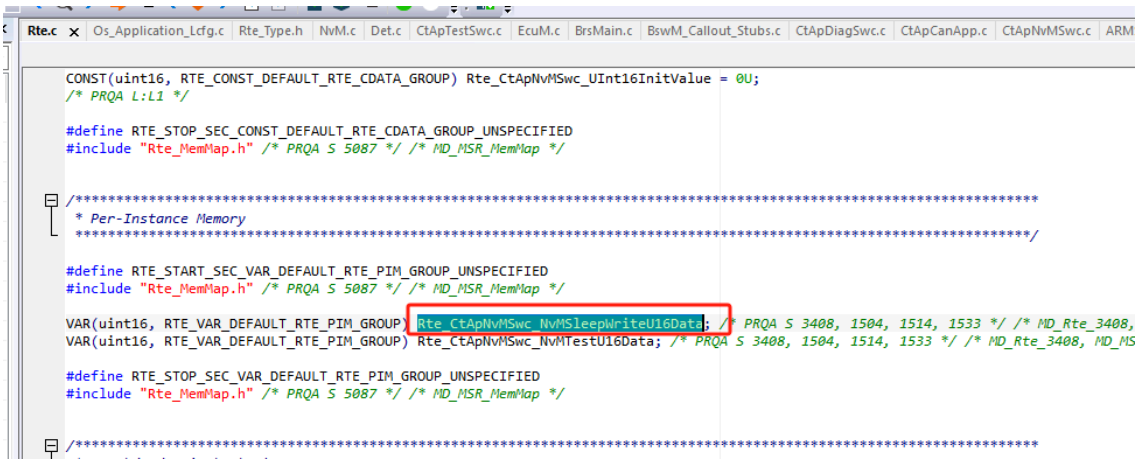


保存，生成全部，再生成swc 代码

2.3、添加测试代码

首先，上电时 NvM_ReadAll 后变量是读在Rte.c 里面的ram变量，后面我们运行过程中，改写整个变量，最后面休眠前调用 NvM_SetRamBlockStatus(NvMConf_NvMBlockDescriptor_CtApNvMSwcNvBlockSleepWriteU16Data, TRUE); ,设置了下电存储的Block ,调用这个接口就会置标志位，代表数据变化了

然后最后调用NvM_WriteAll , 即可实现下电的存储



```
Rte.c | Os_Application_Lcfg.c | Rte_Type.h | NvM.c | Det.c | CtApTestSwc.c | EcuM.c | BrsMain.c | BswM_Callout_Stubs.c | CtApDiagSwc.c | CtApCanApp.c | CtApNvMSwc.c | ARMStartup_CortexM.c

{
/*********************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          << Start of runnable implementation >>          DO NOT CHANGE THIS COMMENT!
 * Symbol: RCtApNvMSwc_Init
 *********************************************************************************************************/
  u16RData = Rte_CtApNvMSwc_NvMTestU16Data;
  u16RData = Rte_CtApNvMSwc_NvMSleepWriteU16Data;
/*********************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          << End of runnable implementation >>          DO NOT CHANGE THIS COMMENT!
 *********************************************************************************************************/
}


#define CtApNvMSwc_STOP_SEC_CODE
#include "CtApNvMSwc_MemMap.h" /* PRQA S 5087 */ /* MD_MSR_MemMap */


/*********************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          << Start of function definition area >>          DO NOT CHANGE THIS COMMENT!
 *********************************************************************************************************/
void RCtApNvMSwc_vSetWriteAllBlock(void)
{
  Rte_CtApNvMSwc_NvMSleepWriteU16Data = 0x2474;
  NvM_SetRamBlockStatus(NvMConf_NvMBlockDescriptor_CtApNvMSwcNvBlockSleepWriteU16Data, TRUE);
  return;
}

/*********************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          << End of function definition area >>          DO NOT CHANGE THIS COMMENT!
```

在用户定义函数区定义一个函数

休眠NvM_WriteAll 前调用我们定义的函数

```
Rte.c | Os_Application_Lcfg.c | Rte_Type.h | NvM.c | Det.c | CtApTestSwc.c | EcuM.c | BrsMain.c | BswM_Callout_Stubs.c | CtApDiagSwc.c | CtApCanApp.c | CtApNvMSwc.c | ARMStartup_CortexM.c
BswM_NvM_WriteAll()

/*********************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          </USERBLOCK>                          DO NOT CHANGE THIS COMMENT!
 *********************************************************************************************************/

} /* End of BswM_INIT_NvMReadAll */


FUNC(void, BSWM_CODE) BswM_NvM_WriteAll(void)
{
/*********************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          <USERBLOCK BswM_NvM_WriteAll>          DO NOT CHANGE THIS COMMENT!
 *********************************************************************************************************/
  NvM_RequestResultType Int_NvMultiBlockStatus = NVM_REQ_NOT_OK;
  Std_ReturnType ret = E_NOT_OK;
  extern void RCtApNvMSwc_vSetWriteAllBlock(void);
  RCtApNvMSwc_vSetWriteAllBlock();
  NvM_WriteAll();

  do
  {
      NvM_MainFunction();
      Fee_MainFunction();
      Fls_MainFunction();
      ret = NvM_GetErrorStatus(NvMConf___MultiBlockRequest, &Int_NvMultiBlockStatus);

  #if defined(VGEN_ENABLE_DRVWD)
    Wdg_Trigger();
  #endif
  } while ((Int_NvMultiBlockStatus == NVM_REQ_PENDING)&&( ret == E_OK ));

/*********************************************************************************************************
 * DO NOT CHANGE THIS COMMENT!          </USERBLOCK>                          DO NOT CHANGE THIS COMMENT!
 *********************************************************************************************************/
```